

Gerenciadores de Layout

Os gerenciadores de layout são fornecidos para organizar componentes GUI em um container para propósitos de apresentação. Eles processam a maioria dos detalhes, fornecendo uma maneira automática de posicionar os componentes gráficos nos containers.

Todos os gerenciadores de Layout implementam a interface `LayoutManager` ou a subinterface `LayoutManager2`. E cada container como um `Painel` ou `Frame` (molduras) possui um gerenciador de layout padrão associado.

É possível desativar o Gerenciador de Layout caso o programador queira definir o tamanho ou posição dos componentes

Para desligar layouts:

`recipiente.setLayout(null);`

Porém torna-se necessário definir posição e tamanho de cada componente:

`componente.setBounds(x, y, larg, alt);`

Assim, utilizando o `setBounds`, podem acabar acontecendo problemas de posicionamento de components quando o tamanho da janela é alterado pelo usuário da aplicação. Pois a posição dos componentes será estática.

Tamanho e Posição dos componentes no Layout

Um container mantém uma referência a uma instância particular do Gerenciador de Layout. Sempre quando for necessário posicionar o componente será invocado o Gerenciador de Layout. Ou seja, se o usuário tentar definir o tamanho ou posição dos componentes (`setSize()` ou `setLocation()`) o Gerenciador de Layout irá anular a decisão.

Cada componente Java tem um tamanho preferencial:

- Normalmente é o menor tamanho necessário para apresentar o componente de maneira visualmente significativa
- Exemplo: o tamanho preferencial de um botão é o tamanho de sua etiqueta de texto.

O Gerenciador de Layout equilibra duas considerações:

- A política de layout e o tamanho preferencial de cada componente.
- A prioridade é para a política de layout

Etapas de criação de um painel

Cada `Painel` é montado em quatro etapas:

- 1 - Construir (instanciar) o painel
- 2 - Dar ao painel o gerenciador de layout
 - a. Quando um container é instanciado (etapa 1) ele recebe um gerenciador de layout padrão. Esta etapa pode ser pulada se o gerenciador a ser utilizado for o padrão

3 - Preencher o painel

a. Envolve montar componentes e acrescentá-los ao painel

4 - Acrescentar o painel ao seu próprio container

Gerenciadores de Layout

FlowLayout

É o gerenciador de layout mais simples, dispõe os objetos sequencialmente da esquerda para a direita na ordem em que foram adicionados

container.setLayout(new FlowLayout());

Permite aos componentes serem alinhados à esquerda, centralizados (padrão) ou à direita:

objetoLayout.setAlignment(FlowLayout.LEFT);

objetoLayout.setAlignment(FlowLayout.RIGHT);

Todos os componentes possuem um método chamado `getPreferredSize()`, usado pelo `FlowLayout` para perguntar o tamanho de cada componente.

Principais métodos:

- `FlowLayout()`: Centraliza os componentes deixando uma distância entre eles (gap) de 5 pixels.
- `FlowLayout(int align)`: Constrói um layout com alinhamento estabelecido (`FlowLayout.CENTER`, `FlowLayout.LEFT`, `FlowLayout.RIGHT`)
- `FlowLayout(int align, int hgap, int vgap)`: Define o alinhamento e a distância horizontal e vertical, respectivamente.

BorderLayout

É o gerenciador padrão para `JFrames`, `JApplets` e `Caixas de Diálogo`. A classe `BorderLayout` herda diretamente da classe `Object` e implementa a interface `LayoutManager2` (uma subinterface de `LayoutManager`)

Organiza os componentes em cinco áreas:

Norte, Sul, Leste, Oeste e Centro.

Os componentes devem ser adicionadas a regiões nomeadas no gerenciador de layout, caso contrário não ficarão visíveis:

• NORTH, SOUTH, EAST, WEST, CENTER

- Norte e Sul têm prioridade sobre Leste e
- Oeste, que têm prioridade sobre Centro
- Se for adicionado um componente ao Centro, este se expande até ocupar todo o

- espaço restante na janela
- O componente colocado em uma região pode ser um container ao qual serão anexados outros componentes (p.ex., JPanel)
- Veja o exemplo BorderLayoutDemo.java

```
layout = new BorderLayout( 5, 5 );
setLayout( layout );
add( buttons[ 0 ], BorderLayout.NORTH ); // add button to north
add( buttons[ 1 ], BorderLayout.SOUTH ); // add button to south
add( buttons[ 2 ], BorderLayout.EAST ); // add button to east
add( buttons[ 3 ], BorderLayout.WEST ); // add button to west
add( buttons[ 4 ], BorderLayout.CENTER ); // add button to center
```

GridLayout

Fornece flexibilidade para colocação de componentes. Divide o container em uma grade de modo que os componentes podem ser colocados em linhas e colunas.

A classe GridLayout herda diretamente da classe Object e implementa a interface LayoutManager.

Com o gerenciador GridLayout, a posição relativa dos componentes não é alterada quando a área é redimensionada. Os componentes são adicionados iniciando na célula na parte superior esquerda da grade e prosseguindo da esquerda para a direita até a linha estar cheia. Então, o processo continua na próxima linha.

Veja o exemplo GridLayoutDemo.java

...

```
gridLayout = new GridLayout( 3, 2 );
setLayout( gridLayout );
add( button );
```

...

BoxLayout

Permite colocar uma única linha ou coluna de componentes com mais flexibilidade do que o GridLayout.

Existe um Container - a classe Box – cujo gerenciador de layout padrão é BoxLayout.

Fornece métodos estáticos para criar um BoxLayout horizontal ou vertical:

- `Box b = Box.createHorizontalBox( );`
 - São organizados da esquerda para direita
- `Box b = Box.createVerticalBox( );`
 - São organizados de cima para baixo
- Depois, adicionam-se os componentes da maneira usual:
 - `b.add(btnOk);`
- Confira o exemplo `TextAreaDemo.java`

SpringLayout

É o Gerenciador de Layout mais flexível que existe. Permite posicionar componentes em posições exatas ou relativas com extrema precisão, porém requer um posicionamento em baixo nível.

Cada componente adicionado ao container deverá ter suas limitações de posicionamento configuradas.

A posição do componente deve ser definida em relação a posição de outro componente já posicionado ou em relação ao container onde está sendo inserido.

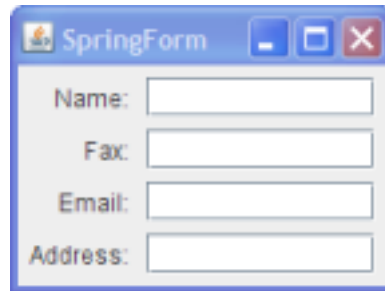
O posicionamento pode ser estabelecido de acordo com as quatro laterais do componente (WEST, EAST, NORTH, SOUTH).

```
layout.putConstraint(SpringLayout.WEST, label, 5, SpringLayout.WEST, this);

layout.putConstraint(SpringLayout.NORTH, label, 5, SpringLayout.NORTH,
this);
```

```
layout.putConstraint(SpringLayout.WEST, textField, 5, SpringLayout.EAST,  
label);
```

```
layout.putConstraint(SpringLayout.NORTH, textField, 5,  
SpringLayout.NORTH, this);
```



Combinando Gerenciadores de Layout

Interfaces complexas exigem que cada componente seja colocado numa localização exata. Para tanto, geralmente usam-se múltiplos painéis, cada um com seu layout específico.

Veja o exemplo `PanelDemo.java`